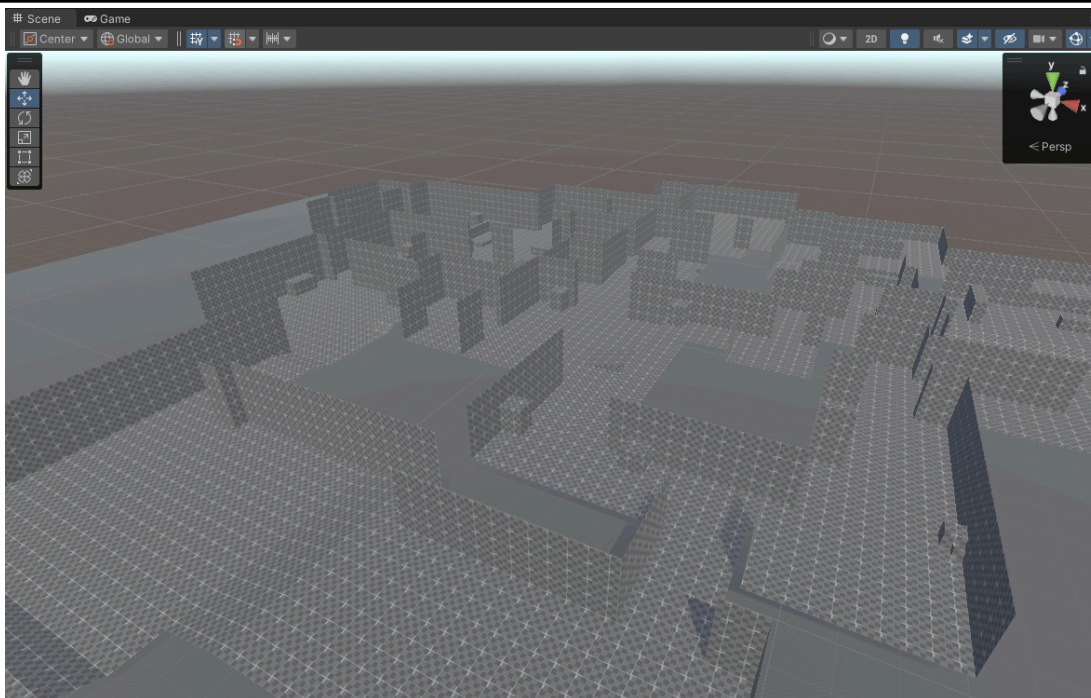


	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación gráfica, visión computacional y multimedia				
TÍTULO DE LA PRÁCTICA:					
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	22/04/2026	HORA DE PRESENTACIÓN			
INTEGRANTE (s): Maxs Sebastian Joaquin Forocca Mamani				NOTA:	
DOCENTE(s): RENE ALONSO NIETO VALENCIA					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Diseño del entorno El entorno 3D desarrollado corresponde a un escenario de combate abierto, para permitir la movilidad del tanque y la interacción con múltiples torretas enemigas. El entorno fue obtenido de un recurso de la Unity Store

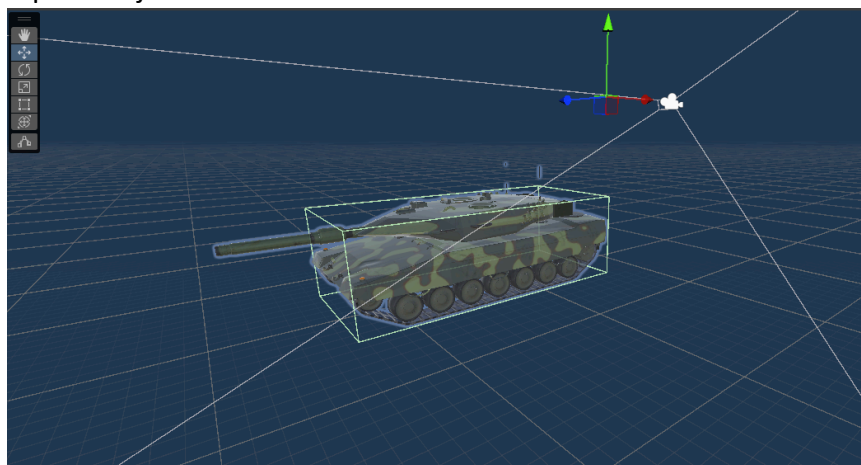


Se prioriza un espacio despejado que facilite la línea de visión y el uso de proyectiles con física, evitando obstáculos que interfieran con las trayectorias de los proyectiles.

2. Personaje Controlable (Tanque)

El personaje controlable es un Asset 3D que usa el modelo de un tanque 3D que permite controlar la torreta y el cañón, este recurso fue obtenido de la Unity Store.

Adicionalmente el Main Cámara se movió dentro de la torreta para tener la perspectiva de tercera persona al mover el personaje



2.1 Movimiento y rotación

El tanque se controla mediante entrada del usuario usando el sistema de ejes de Unity:

```
1 reference
49 void HandleMovement()
50 {
51     float move = Input.GetAxis("Vertical"); // W/S
52     float turn = Input.GetAxis("Horizontal"); // A/D
53
54     // Movimiento adelante/atrás
55     transform.Translate(0,0, move * moveSpeed * Time.deltaTime);
56
57     // Rotación del tanque
58     transform.Rotate(0, turn * rotationSpeed * Time.deltaTime,0);
59 }
60
```

- Translate permite el desplazamiento hacia adelante/atrás
- Rotate controla la rotación sobre el eje Y
- Time.deltaTime asegura independencia del framerate

2.2 Control de torreta y cañón

La torreta rota con el mouse (eje horizontal), el cañón se eleva con teclas (Q/E) y se limita el ángulo mediante Mathf.Clamp

```
70 void HandleCannonRotation()
71 {
72     float input = 0f;
73
74     if (Input.GetKey(KeyCode.E)) input = -1f; // subir
75     if (Input.GetKey(KeyCode.Q)) input = 1f; // bajar
76
77     // Ajustar angulo
78     currentCannonAngle += input * cannonSpeed * Time.deltaTime;
79
80     // Limita el rango de angulo de rotacion del cañón
81     currentCannonAngle = Mathf.Clamp(currentCannonAngle, minCannonAngle, maxCannonAngle);
82
83     // Aplicar rotacion local en X
84     cannon.localRotation = Quaternion.Euler(currentCannonAngle, 0f, 0f);
85 }
```

- Restricción de Ángulos: Utiliza un acumulador currentCannonAngle limitado por Mathf.Clamp para evitar que el cañón rote más allá de los límites mecánicos del diseño.
- cannonSpeed controla qué tan rápido gira (verticalmente).

- localRotation significa que la rotación se hace respecto al padre del objeto, no al mundo global.
- Quaternion.Euler(...) convierte los ángulos en una rotación usable por Unity. Para el cañón se usa solo el eje X.

2.3 Sistema de disparo

Se implementó un sistema de disparo con carga o velocidad inicial de disparo oscilante básica:

- HandleShooting(): Mientras se mantiene presionado el click izquierdo, se calcula una velocidad de salida que oscila entre un valor mínimo y máximo.

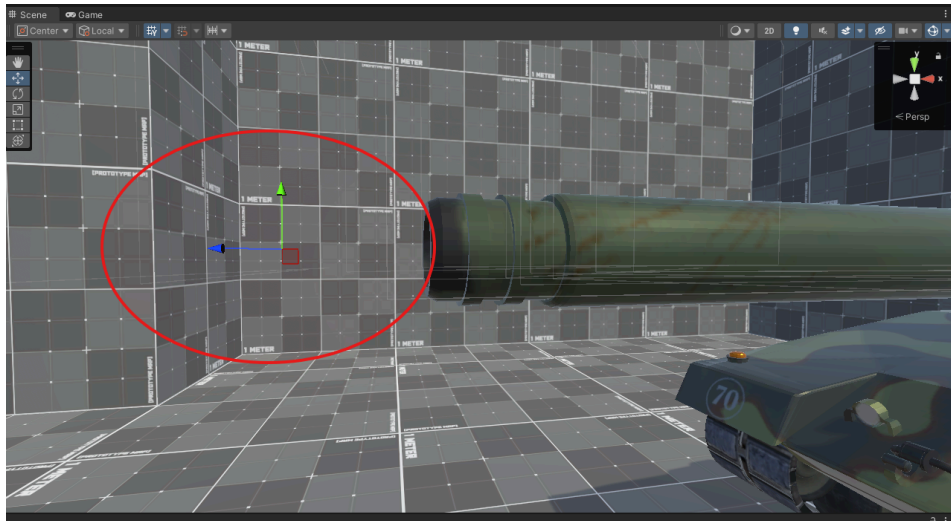
```
100 // Mantener click → cargar
101 if (Input.GetMouseButton(0))
102 {
103     isCharging = true;
104     chargeTime += Time.deltaTime;
105 }
```

- Oscilación Senoidal: Utiliza Mathf.Sin junto con Mathf.Lerp para que la potencia suba y baje cíclicamente, obligando al jugador a soltar el botón en el momento preciso para alcanzar la distancia deseada.

```
106 // Oscilación tipo seno
107 float t = (Mathf.Sin(chargeTime * oscillationSpeed) + 1f) / 2f;
108
109 currentMuzzleVelocity = Mathf.Lerp(minVelocity, maxVelocity, t);
110 trajectory.ShowTrajectory(currentMuzzleVelocity);
```

- Shoot(float velocity): Instancia el proyectil y le asigna una velocidad inicial basada en el vector frontal (forward) del punto de disparo multiplicado por la potencia cargada.

```
136 // Proyectil
137 GameObject bullet = Instantiate(bulletPrefab, firePoint.position, firePoint.rotation);
138 AudioManager.Instance.PlaySFX(shootSound);
139 Rigidbody rb = bullet.GetComponent<Rigidbody>();
140
141 if (rb != null)
142 {
143     rb.velocity = firePoint.forward * velocity;
144 }
145 }
```



3. Sistema de Projectiles y Física

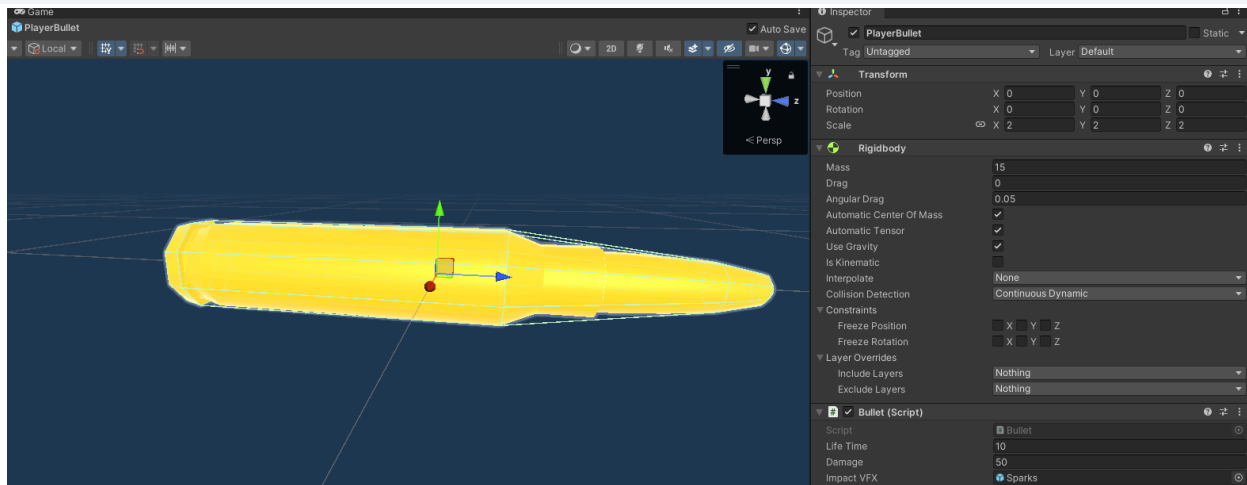
El sistema de proyectiles utiliza el componente Rigidbody de Unity, permitiendo simular comportamiento físico realista.

3.1 Configuración Clave

- Use Gravity: activado
- Collision Detection: Continuous Dynamic
- Velocidad inicial asignada directamente

C/C++

```
rb.velocity = firePoint.forward * muzzleVelocity;
```



3.2 Projectiles

Se usaron Assets 3D de la Unity Store para el proyectil.

- Start(): Utiliza Destroy(gameObject, lifeTime) para asegurar que el proyectil se elimine después de un tiempo determinado (10 segundos por defecto). Esto es vital para el rendimiento del sistema y evitar fugas de memoria.

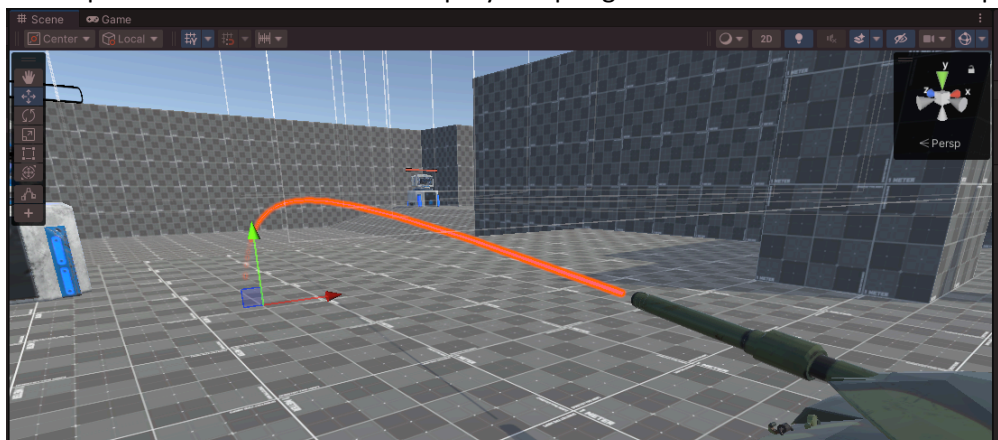
3.3 Trayectoria

Para mejorar la precisión del jugador, se incluyó un predictor de trayectoria parabólica.

```

26         for (int i = 0; i < steps; i++)
27         {
28             float t = i * timeStep;
29
30             // Fórmula física: posición = p0 + v*t + (1/2) g t²
31             Vector3 point = startPosition +
32                 startVelocity * t +
33                 0.5f * Physics.gravity * t * t;
34
35             line.SetPosition(i, point);
36         }
37
38         line.enabled = true;
    
```

- ShowTrajectory(float velocity): Utiliza un LineRenderer para dibujar el camino que seguirá la bala.
- Cálculo Cinemático: Aplica la fórmula de posición en un movimiento uniformemente acelerado. Esto permite visualizar la caída del proyectil por gravedad antes de realizar el disparo.



4. Gestión de Colisiones

Las colisiones están enfocadas en los proyectiles

OnCollisionEnter(Collision collision): Se activa al detectar un impacto físico.

```

17 void OnCollisionEnter(Collision collision)
18 {
19     if (impactVFX != null && collision.contacts.Length > 0)
20     {
21         ContactPoint contact = collision.contacts[0];
22
23         Quaternion rotation = Quaternion.LookRotation(contact.normal);
24
25         GameObject vfx = Instantiate(impactVFX, contact.point, rotation);
26         Destroy(vfx, 2f);
27     }
28     // Colisión con el enemigo, verificando si tiene componente Health
29     Health health = collision.gameObject.GetComponentInParent<Health>();
30
31     if (health != null)
32     {
33         health.TakeDamage(damage);
34         GameManager.Instance.AddScore(1);
35     }
36
37     // Destruye la bala SIEMPRE al colisionar
38     Destroy(gameObject);
39 }

```

- VFX de Impacto: Calcula el punto de contacto y la normal de la superficie para instanciar un efecto visual (impactVFX) orientado correctamente.
- Aplicación de Daño: Busca el componente Health en el objeto impactado. Si lo encuentra, llama a TakeDamage().
- Diferenciación: En Bullet.cs, si se impacta a un enemigo con éxito, se comunica con el GameManager para sumar puntos al puntaje global.

5. Mecánicas

5.1 Mecánicas de los Enemigos (EnemyTurretController.cs)

Este script dota de inteligencia artificial básica a las torretas del entorno, permitiéndoles rastrear y atacar al jugador.

- Update(): Controla el ciclo de vida de la torreta. Verifica si el juego está pausado, si el jugador existe y si la torreta aún tiene vida antes de calcular la distancia al objetivo.

```

39 void Update()
40 {
41     if (PauseManager.Instance != null && PauseManager.Instance.IsPaused)
42         return;
43     if (player == null) return;
44     if (health != null && health.isDead) return;
45     float distance = Vector3.Distance(transform.position, player.position);
46
47     if (distance <= detectionRange)
48     {
49         RotateTurret();
50         AimCannon();
51     }

```

- RotateTurret(): Maneja la rotación horizontal (Eje Y). Usa Quaternion.Slerp para que el giro hacia el jugador sea suave y no instantáneo.

```

60 // Rotación horizontal (torreta)
61 1 reference
62 void RotateTurret()
63 {
64     Vector3 direction = player.position - turret.position;
65     direction.y = 0f; // ignorar altura
66     Quaternion targetRotation = Quaternion.LookRotation(direction);
67     turret.rotation = Quaternion.Slerp(
68         turret.rotation,
69         targetRotation,
70         turretRotationSpeed * Time.deltaTime
71     );
72 }

```

- AimCannon(): Controla la elevación vertical. Incluye una normalización de ángulos y un Mathf.Clamp (entre -10° y 30°) para evitar que el cañón gire de forma poco realista o atravesase el modelo 3D de la torreta.

```

75 void AimCannon()
76 {
77     Vector3 direction = player.position - cannon.position;
78     Quaternion targetRotation = Quaternion.LookRotation(direction);
79     Vector3 angles = targetRotation.eulerAngles;
80
81     // Normalizar ángulo X
82     float xAngle = angles.x;
83     if (xAngle > 180f) xAngle -= 360f;
84
85     // Limitar elevación (opcional)
86     xAngle = Mathf.Clamp(xAngle, -10f, 30f);
87
88     Quaternion finalRotation = Quaternion.Euler(xAngle, turret.eulerAngles.y, 0f);
89
90     cannon.rotation = Quaternion.Slerp(
91         cannon.rotation,
92         finalRotation,
93         cannonRotationSpeed * Time.deltaTime
94     );
95 }
96

```

- Shoot(): Instancia el prefab de la bala enemiga y le asigna una velocidad inicial a través de su Rigidbody en la dirección frontal del cañón.

```

99 void Shoot()
100 {
101     // VFX en FirePoint
102     if (muzzleFlashPrefab != null)
103     {
104         GameObject vfx = Instantiate(muzzleFlashPrefab, firePoint.position, firePoint.rotation);
105         Destroy(vfx, 2f); // limpieza automática
106     }
107     AudioManager.Instance.PlaySFX(shootSound);
108     GameObject bullet = Instantiate(
109         bulletPrefab,
110         firePoint.position,
111         firePoint.rotation
112     );
113
114     Rigidbody rb = bullet.GetComponent<Rigidbody>();
115
116     if (rb != null)
117     {
118         rb.velocity = firePoint.forward * muzzleVelocity;
119     }
120 }

```

6. Enlace a github: <https://github.com/MaxsForocca/LabComputacionGrafica>

7. Video Drive:

https://drive.google.com/file/d/18mN1WnAvCT9LkKrv1F0bXpn7Q3qC_eQ/view?usp=sharing

II. SOLUCIÓN DEL CUESTIONARIO

III. CONCLUSIONES

La realización de este laboratorio permitió la implementación efectiva de un motor de física y sistemas de detección de colisiones dentro de un entorno 3D, consolidando habilidades críticas en el desarrollo de videojuegos con Unity. A través de la integración de mecánicas cinemáticas para el control del tanque, la simulación precisa de trayectorias parabólicas mediante fórmulas físicas, y la lógica de combate soportada por una inteligencia artificial básica para torretas enemigas, se logró construir un sistema interactivo y coherente. El proyecto no solo destacó por la aplicación técnica de Rigidbodies y el cálculo de trayectorias, sino también por un enfoque consciente en la optimización del rendimiento y la gestión del ciclo de vida de los objetos, resultando en un entorno funcional que equilibra la fidelidad física con una jugabilidad fluida y bien estructurada.



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLE-001

Página: 10

REFERENCIAS Y BIBLIOGRAFÍA